

Table of Contents

VLLM: Kimi-K3 MXFP4 Benchmark Guide for AMD Instinct MI355X 1

 Step 1: Set MI355X vLLM ROCm image 1

 Step 2: Hugging Face model download 1

 Step 3: Prepare benchmarking files and run measurements 1

 Create the server and client files 2

 Create the main script to loop over all test configurations 4

 Benchmark Results..... 6

VLLM: Kimi-K3 MXFP4 Benchmark Guide for AMD Instinct MI355X

There are three steps to reproduce our measurements on vLLM on MI355X. First, we pull a Docker image with recent vLLM commits. Second, we download the model weights from the HuggingFace repository. Third, we prepare the benchmarking files and run measurements.

Step 1: Pull the MI355X vLLM ROCm image

Download the vLLM v0.29.0 tagged release container:

```
docker pull vllm/vllm-openai-rocm:v0.29.0
```

Use this container for benchmarking:

```
export MI355X_IMAGE=vllm/vllm-openai-rocm:v0.29.0
```

Step 2: Hugging Face model download

Download link: [moonshotai/Kimi-K3 · Hugging Face](#)

You can initiate the download with:

```
pip install huggingface_hub
HF_XET_HIGH_PERFORMANCE=1 \
HF_HOME=<hf_home> \
HF_TOKEN=<hf_token> \
hf download moonshotai/Kimi-K3 --exclude "original/*" --exclude "metal/*"
```

Step 3: Prepare benchmarking files and run measurements

Create the benchmark files

Create **start_server.sh** in the current directory. This is used to launch the vLLM server inside the container.

start_server.sh

```
#!/usr/bin/env bash
set -euo pipefail
```

```

# — Hardcoded Configuration —————
HF_CACHE_ROOT="/data/huggingface-cache"
SNAPSHOT_PATH="<path to snapshots>"
HF_TOKEN="<hf_token>"
MI355X_IMAGE="vllm/vllm-openai-rocm:v0.29.0"
RESULTS_DIR="<path to results>"
CONTAINER_NAME="kimi-k3-server"
TP=8
# —————
mkdir -p "$RESULTS_DIR"
docker rm -f "$CONTAINER_NAME" 2>/dev/null || true
echo "Starting vLLM server container..."
docker run -d \
  --name "$CONTAINER_NAME" \
  --ipc=host --network=host --privileged --shm-size=16g \
  --device=/dev/kfd --device=/dev/dri --device=/dev/mem \
  --cap-add=SYS_PTRACE --security-opt seccomp=unconfined \
  -v "${HF_CACHE_ROOT}:/hf_cache" \
  -v "${RESULTS_DIR}:/results" \
  -e HF_TOKEN="${HF_TOKEN}" \
  -e HF_HUB_OFFLINE=0 \
  -e VLLM_ROCM_USE_AITER=1 \
  -e SAFETENSORS_FAST_GPU=1 \
  -e VLLM_ROCM_USE_AITER_MOE_SITUV2_A8W4=1 \
  -e AITER_BF16_FP8_MOE_BOUND=0 \
  -e VLLM_USE_BREAKABLE_CUDAGRAPH=0 \
  -e VLLM_USE_RUST_FRONTEND=0 \
  -e VLLM_ENGINE_CORE_INIT_TIMEOUT=2400 \
  --entrypoint bash \
  "${MI355X_IMAGE}" \
  -c "vllm serve /hf_cache/${SNAPSHOT_PATH} \
    --served-model-name moonshotai/Kimi-K3 \
    --host 0.0.0.0 \
    --port 8000 \
    --trust-remote-code \
    --tensor-parallel-size ${TP} \
    --load-format auto \
    --gpu-memory-utilization 0.95 \
    --mm-encoder-tp-mode data \
    --max-num-seqs 128 \
    --max-num-batched-tokens 4096 \
    --compilation-config.cudagraph_mode FULL_DECODE_ONLY \
    --compilation-config.custom_ops+ +fused_rms_norm_gated \
    --reasoning-parser kimi_k3"

echo ""

```

```

echo "Container '$CONTAINER_NAME' started."
echo "Waiting for server to be ready..."
echo "Watch logs with: docker logs -f $CONTAINER_NAME"
echo ""

TIMEOUT=2400
ELAPSED=0
while true; do
    if docker logs "$CONTAINER_NAME" 2>&1 | grep -q "Application startup complete"; then
        echo ""
        echo "✓ Server is ready! Now run: ./run_benchmarks.sh"
        break
    fi

    STATUS=$(docker inspect -f '{{.State.Status}}' "$CONTAINER_NAME" 2>/dev/null)
    if [ "$STATUS" != "running" ]; then
        echo ""
        echo "ERROR: Container stopped. Full logs:"
        docker logs "$CONTAINER_NAME" 2>&1
        exit 1
    fi

    if [ "$ELAPSED" -ge "$TIMEOUT" ]; then
        echo "ERROR: Server did not start within ${TIMEOUT}s"
        docker logs "$CONTAINER_NAME" 2>&1 | tail -40
        exit 1
    fi

    sleep 30
    ELAPSED=$((ELAPSED + 30))
    echo " ...still loading (${ELAPSED}s elapsed)"
done

```

Create the script to loop over all test configurations

Prepare a script called **run_benchmarks.sh** in the current directory with the following commands. This file will loop over all the test configurations to:

1. Wait until the server is ready.
2. Launch the client and perform measurements.
3. Output the benchmark files to a results directory.

run_benchmarks.sh

```

#!/usr/bin/env bash
set -euo pipefail

# — Configuration —————
CONTAINER_NAME="kimi-k3-server"

```

```

SERVED_MODEL_NAME="moonshotai/Kimi-K3"

# Add more entries to test multiple ISL/OSL pairs
ISL_OSL_LIST=(
    "128000,1000"
)

CONC_LIST=(1 2 4 8 16 32 64 128)

# — Pre-flight check —————
STATUS=$(docker inspect -f '{{.State.Status}}' "$CONTAINER_NAME" 2>/dev/null || echo
"missing")
if [ "$STATUS" != "running" ]; then
    echo "ERROR: Server container '$CONTAINER_NAME' is not running."
    echo "  Run ./start_server.sh first."
    exit 1
fi

if ! docker logs "$CONTAINER_NAME" 2>&1 | grep -q "Application startup complete"; then
    echo "ERROR: Server does not appear ready yet."
    echo "  Wait for start_server.sh to report ready, then re-run."
    exit 1
fi

echo "✓ Server is running and ready"
echo ""

# — Benchmark loop —————
TOTAL=$(( ${#ISL_OSL_LIST[@]} * ${#CONC_LIST[@]} ))
COUNT=0

for P in "${ISL_OSL_LIST[@]"; do
    ISL=${P%,*}
    OSL=${P#*,}

    for C in "${CONC_LIST[@]"; do
        COUNT=$((COUNT + 1))
        echo "—————"
        echo "  Run ${COUNT}/${TOTAL}: ISL=${ISL} OSL=${OSL} CONC=${C}"
        echo "—————"

        NUM_PROMPTS=$((C * 10))

        docker exec \
            -e ISL="$ISL" \
            -e OSL="$OSL" \

```

```

-e CONC="$C" \
-e SERVED_MODEL_NAME="$SERVED_MODEL_NAME" \
"$CONTAINER_NAME" \
vllm bench serve \
  --host localhost \
  --port 8000 \
  --endpoint /v1/completions \
  --model "$SERVED_MODEL_NAME" \
  --dataset-name random \
  --random-input-len "$ISL" \
  --random-output-len "$OSL" \
  --random-range-ratio 0.2 \
  --ignore-eos \
  --max-concurrency "$C" \
  --num-prompts "$NUM_PROMPTS" \
  --percentile-metrics ttft,tpot,itl,e2el \
  --metric-percentiles 25,50,75,90,95,99 \
  --trust-remote-code \
  --save-result \
  --result-dir /results \
  --result-filename "Kimi-K3-MXFP4_isl${ISL}_osl${OSL}_c${C}.json"

  echo "  ✓ Done – result saved to /results/Kimi-K3-
MXFP4_isl${ISL}_osl${OSL}_c${C}.json"
  echo ""
done
done

echo "All ${TOTAL} benchmarks complete."
echo "Results are in: ${RESULTS_DIR:-$(pwd)/results}"

```

Run the workflow

```

chmod +x start_server.sh run_benchmarks.sh
./start_server.sh
./run_benchmarks.sh

```

Benchmark Results

TTPS/GPU = total_token_throughput / TP, where TP=8

Config	Concurrency	MI355X vLLM (TTPS/GPU)
128K/1K	1	874
128K/1K	2	1090
128K/1K	4	1235
128K/1K	8	1199
128K/1K	16	1179
128K/1K	32	1118
128K/1K	64	1096
128K/1K	128	1110